

CSSO, Vid 1
(using VS code)

enter: `python --version`
to check what version of python
I am working with

Terminal window
- Command line interface (CLI)

Python is a programming language

User input is detected, received, and interpreted there

type: `Code filename.py` (replace "filename" with preferred name)
any program I write in python has a file name that ends in .py
to indicate to the computer that it's a program written in python
also type: `python filename.py` to run the program of choice.

My interface, or a users' interface, is the CLI; the terminal. In programming, the CLI is the only way to run a program.

In the VS code application that I am using, there is the text:

`PS C:\Users\John\workspace>`

This line indicates my prompt: where the CLI wants me to input commands.

My prompt indicator shows information of the file I am working with:

also known as the "PS command" (PS C)

- determines the file name and arguments that are given when the process was created by memory

To run a file, type: `python filename.py`

this runs the program in the selected file. The computer detects things from top to bottom, left to right. It sees "python" first, indicating the type of language used, and then "filename.py", the file / program I want to initiate.

Computers only understand the binary system: 0 and 1.

Python is not only a language used to write code. It is also a program, known as an interpreter that translates words into the binary system.

CTRL keys:

- CTRL + O : opens an existing file
- CTRL + ' : opens the terminal
- CTRL + L : clears the terminal
- CTRL + J : closes the terminal

In most programming language, we have Functions. Functions are actions that let the user do something in a program; something the computer knows how to do.

Arguments are inputs to functions that influences its behavior. They are necessary to help a function do what it is supposed to do.

What the program is doing on a screen is a Side Effect. It is the representation of what a function has done or is doing, whether it be visual or audio. It is explicit to users.

Mistakes in a program are called Bugs. Usually take the form of something we did not write correctly and are necessary to fix in order to run a program properly.

ex. the print function

`print()` ← "print" is a function. It lets us tell the computer to print something.

`Print("Hello, World")` ← "Hello, World" is an argument. It is an input to the "print" function that tells the computer to print "Hello, World".

in terminal: Hello, World ← The presented text "Hello, World" is a Side effect. The visual display of what the computer did.

assume: `print("Hello, World"` ← Closing parenthesis is missing, representing a bug: something that needs to be fixed. For this example, this is a Syntax Error: a keyboard error.

another function we use: input

ex. 1. print("What is your name? ")	What is your name? [user input]
2. input()	hello, world
3. print("hello, world")	

Since the "input" function takes in an argument, we can remove the first line of text entirely and replace it with the "input" function.

1. input("What is your name? ")	What is your name? [user input]
2. print("hello, world")	hello, world

We Hardcoded "hello, world," so the users' input does not matter. To apply the users' input to the side effect, or the outcome of the code, we use a feature of some functions, Return Values.

essentially:

get users' input. → hand it back so that the programmer can do stuff with it.

To acquire a return value, we need a Variable. It is the computer's capability of creating a memory, where such a variable holds and stores a value. A variable is just a container for some value inside of a computer/program.

Variables in programming is not restricted to $x, y, z, \dots$ We can use words to describe variables. For this example, assume "x" is the name of the variable.

x = input("What is your name? ")	The input that the computer receives i.e. the input we present becomes stored in the variable "x".
----------------------------------	--

In coding, the equal sign "=" doesn't mean equality. It means assignment (the equal sign is the assignment operator).

The equal sign copies from right to left, whatever the return value of the function on the right is.

Since the `=` sign "assigns" a variable to a certain input, the computer stores that input in the assigned variable, like a container holding information.

The variable is then used in various functions to explicitly show the input to the user.

1. `x = input("what is your name? ")` | what is your name? [user input]
2. `print("hello," + x)` | hello, [user input]

Comments are notes to self in the code/file. These are not run by the program and are ignored completely as to avoid crashing the code.
ex. `# this is a note to self. The computer basically does nothing with this.`

Pseudocode is NOT python, nor is it English, but allows the user to outline their plans for programming. There is essentially NO code at all. Only comments are present in the file, providing an overview of the users' plans.
ex. `# ask users' name`
`# present name after saying hello`

To improve the code so that it is grammatically correct (`hello, <space> [user input]`), we can fix it in multiple ways. Here are a couple:

- `print("hello," + x)` ← add a space within the quotations (hard coding)
- `print("hello," x)` ← second comma outside of quotations allows some functions to take multiple arguments/inputs.

When passing multiple arguments to print, it automatically inserts a space (see PPS.doc).
The difference between the two methods:

- | | |
|--|---|
| <code>print("hello," + x)</code> | <code>print("hello," x)</code> |
| - one large argument presented by the print function | - two different arguments, both presented by the print function |

The plus `+` is used to imply concatenation; joining the thing on the left to the thing on the right.

If we wanted to concatenate multiple Strings in a single function, we can keep using the '+' operator.

ex. assume variables $x, y, z, \dots$ have values
`print("hello," + x + y + z + w)`

Str is a technical name for "String." It is a type of data that represents text. Note, in most cases, the user needs to specify whether a certain value is a Str or not, notably, numbers. For example, 1 is a number. If we do not specify 1 as an integer, or int, the computer identifies the number "1" as text.

- RR5.doc goes more in depth about function parameters, but for now, focus on `end="\n"` and `Sep=" "`.

ex.

<code>Print("hello,")</code>		hello
<code>print("world")</code>		World

if we add a second argument including `end=""`, we get:

<code>print("hello," end="")</code>		helloworld
<code>print("world")</code>		

Whatever is in the quotations is what gets put at the end. Since there is nothing in the quotations, the first "print" function puts nothing at the end of its function and continues onto the next "print" function.

`Sep=" "` works similarly. Whatever is in the quotations becomes what each argument will be separated by.

ex. assume variable x holds some floating value,
`print("hello," x, sep="?!")`

		hello,?!x
--	--	-----------

`end=` and `Sep=` are both named parameters, because they are both optional and you can pass them in at the end of the print statement.

normal arguments are called positional parameters.

To print quotations:

`print("hello, "friend")` ← this is invalid syntax
`print('hello, "friend"')` → hello, "friend"

the above code shows that we need to specify what type of quotations we actually want to print out.

We can also use backslash (\) because they are Escape Characters.

`print("hello, \"friend\"")` → hello, "friend"

Format String (FString)

- tells python to format stuff in the string in a special way

ex. asking for name

<code>x = input("What's your name? ")</code>		What's your name? [user input]
<code>print(f"hello, {x}")</code>		hello, [user input]

Strings (Str) themselves hold a lot of built-in functionality. Str's are a data-type, a type of data in python which means users can do stuff with Str's.

ex.

<code>x = input("What's your name? ")</code>		What's your name? [user input]
<code>x = x.strip()</code>		# assume [user input]: _____ poob _____
<code>print(f"hello, {x}")</code>		hello, poob

x is the variable name

If "x" is a Str, we can access functionalities that come with Str's by putting:
<Variable name>.<desired function(>> Nothing in parentheses

Using this function, users can change the value of variables using the "=".

In `x = x.strip()`, `x.strip()` returns to me via a return value, but with no spaces to the left or right of the value, updating and changing the value inside the x variable.

documentation states that `.strip()` removes the white space on the left and right of a given variable. `.lstrip()` removes only the left, `.rstrip()` removes only the right.

another function used on str, much like `.strip()` is `.capitalize()`.

ex. assume [user input]: poob eater

<pre>x = input("What's your name? ") x = x.strip() x = x.capitalize() print(f"hello, {x}")</pre>	<pre>What's your name? [user input] hello, Poob eater</pre>
--	---

Another function used on str is `.title()` and we can use it to capitalize the "e" in "eater" from the previous example.

ex. assume [user input]: poob eater

<pre>x = input("What's your name? ") x = x.strip() x = x.title() print(f"hello, {x}")</pre>	<pre>What's your name? [user input] hello, Poob Eater</pre>
---	---

The functions added to the str variables can be chained together. For example:

<pre>x = x.strip() x = x.title()</pre>	<pre>x = x.strip().title()</pre>
2 lines of code	1 line of code

This can be further simplified because there exists an equal sign, reading the code from right to left. For example:

<pre>x = input("What's your name? ") x = x.strip() x = x.title()</pre>	<pre>x = input("What's your name? ").strip().title()</pre>
3 lines of code	1 line of code

This type of simplification is not so much a necessity as it is a preference.

using `.split()`

- this function splits a variable into two smaller substrings depending on the existence of a (space).
- because split is going to return a sequence of values in the form of strings, we need to assign variables to those smaller substrings.

ex. Assume [user input]: poob enter

```
x = input("What's your name? ").strip().title()
first, last = x.split(" ")
print(f"hello, {first}")
```

What's your name? [user input]
hello, Poob

Going to another data type: int (short for integer)

When using integers, python can take the usual arithmetic operators and apply them.

+ (addition) - (subtraction) * (multiplication) / (division)

the percent sign (%) is a modulo operator that allows users to take the remainder after dividing one number by another.

* We do not need to write code in a file and run it in a terminal window.

Interactive mode

- just start writing code and run it immediately.
- The interactive mode is used to access python immediately just for the sake of convenience.

in the terminal window, type: python

we see this:

```
>>> ← this space is where the user types stuff
```

The triple brackets show that it is in the interactive mode for python.

we can do this:

```
>>> 1 + 1
```

```
2
```

```
>>> print("hello, world")
```

```
hello, world
```

```
>>>
```


Mathematics in python

ex.

```
x = 1
y = 2
z = x + y
print(z)
```

here, we see that python is able to detect these numbers "1" and "2" and assign them to their respective variables. Variable "z" utilizes the aforementioned variables to add their values, and finally, "z" is printed.

ex. assume user inputs [x]: 1 and [y]: 2

```
x = input("what's x? ")
y = input("what's y? ")
z = x + y
print(z)
```

what's x? [x]
what's y? [y]
12

Here, we see that python identifies the users' inputs as str, not int. Essentially, the values we provided (1 and 2) are a different data type, and since python interprets them as strs, it concatenates the texts 1 and 2 and puts them together.

To convert user input from str to int, we use the `int()` function. These values we plug in must look like an integer in order to pass through the `int()` function.

no
a, b, c, l, 0

yes
0, 1, -1, 2, 5, 23

ex. assume user inputs [x]: 1 and [y]: 2

```
x = input("what's x? ")
y = input("what's y? ")
z = int(x) + int(y)
print(z)
```

what's x? [x]
what's y? [y]
3

we can simplify the previous example by nesting functions within functions.

ex. assume user inputs $[x]: 1$ and $[y]: 2$

<code>x = int(input("what's x? "))</code>	what's x? $[x]$
<code>y = int(input("what's y? "))</code>	what's y? $[y]$
<code>print(x + y)</code>	3

again, this change between this example and the previous example is less of a necessity but more of a preference. HOWEVER, we must be aware of the trade-offs that exist between the two. Readability is one of them.

ex. further simplifying the above:

`print(int(input("what's x? ")) + int(input("what's y? ")))`

pro:

this is just one single line of code.

con:

long as fuck.

float

- another datatype, namely a number with a decimal part called a floating point value.

ex. assume user inputs $[x]: 1.2$ and $[y]: 3.4$

<code>x = float(input("what's x? "))</code>	what's x? $[x]$
<code>y = float(input("what's y? "))</code>	what's y? $[y]$
<code>print(x + y)</code>	4.6

If we simply want to round the printed value to the nearest integer, the function "round()". (see RPS.doc for more info about documentation)

ex. assume user inputs $[x]: 1.2$ and $[y]: 3.4$

<code>x = float(input("what's x? "))</code>	what's x? $[x]$
<code>y = float(input("what's y? "))</code>	what's y? $[y]$
<code>z = round(x + y)</code>	5
<code>print(z)</code>	

If integers get too big, we want to add commas every 3 digits.
for example, 100,000,000,000,000

In python, we use the f string again to format integers to include commas.

ex. assume user inputs [x]: 999 and [y]: 1

x = float(input("what's x? "))

y = float(input("what's y? "))

z = round(x/y) # round is not necessary here

print(f"{z}")

What's x? [x]

What's y? [y]

1,000

previously, if we never put in the f string, we would have gotten
1000 and not 1,000.

When so many digits are visible, it is an inevitability of using floats.

There are no upper bounds to how large an int value can be, but
there is a bound on how precise a floating point value can be.

ex. assume user inputs [x]: 2 and [y]: 3

x = float(input("what's x? "))

y = float(input("what's y? "))

z = x/y

print(z)

What's x? [x]

What's y? [y]

0.6666666666666666

does not go on infinitely

applying round:

x = float(input("what's x? "))

y = float(input("what's y? "))

z = round(x/y, 2)

print(z)

What's x? [x]

What's y? [y]

0.67

rounds to 2nd decimal digit

applying f string

x = float(input("what's x? "))

y = float(input("what's y? "))

z = x/y

print(f"{z:.2f}")

What's x? [x]

What's y? [y]

0.67

"Inventing" our own functions (def)

ex. assume user behaves "hello()" function exists

```
name = input('what's your name? ') | what's your name? [user input]
hello()
print(name) | NameError: name 'hello' is not defined
```

def is short for define

ex. `def hello():`

Python is now going to treat every line of code that is indented underneath the function `hello` as its meaning.

Parentheses with nothing inside "()" means the function is not taking any inputs or arguments yet. Colon ":" means look for indentation. Everything indented and underneath the defined code is a part of the function.

ex. assume [user input] = poob

```
def hello(): | what's your name? [user input]
    print(hello) | hello
name = input('what's your name? ') | poob
hello()
print(name)
```

Parameterizing a defined function

ex. assume "to" is the chosen parameter

```
def hello(to): | what's your name? [user input]
    print("hello", to) | hello, poob
name = input("what's your name? ")
hello(name)
```

1st and 2nd line defines the function "hello", but the "hello" function is designed to take a single parameter as input and using the value of that parameter (to) to plug into "print" to see not only hello, but the person's name.

In line 4, "hello" function is not only called, but the variable name is passed as input, as an argument, which is passed into "hello".

Essentially, variable "name" becomes "to".

We can set default values to parameters if the user does not input an argument

ex. `def hello(to="world"):`
 `print("hello, ", to)`
`hello()`
 `name = input("what's your name? ")`
 `hello(name)`

hello, world
what's your name? [user input]
hello, poob

Since python's interpreter reads things top bottom, the function "hello" or any defined function of choice must be defined first, then called. If I want a defined function and its components at the bottom of its lines, I need to define another function that holds the function mentioned before.

After that I must call the second defined function to actually run it.

ex. assume "main" is the second function that holds the "hello" function

`def main():`
 `name = input("what's your name? ")`
 `hello(name)`
`def hello(to="world"):`
 `print("hello, ", to)`
`main()`

what's your name? [user input]
hello, poob

It is important to note that "hello" is defined as taking an argument "to" and then I passed the value of the variable "name" into "hello".

`def hello(to="world"):` ← "hello" is defined (to is a parameter)
 `print("hello, ", to)` ← component of "hello" takes in an argument "to"

`def main():` ← function "main" is defined (no parameters)
 `name = input("what's your name? ")` ← variable "name" takes and stores input
 `hello(name)` ← value of variable "name" is put into function "hello" as a variable

This is why "main" is listed before "hello".

Assume we make a mistake:

```
ex. def main():  
    name = input("What's your name? ")  
    hello()  
  
def hello():  
    print("hello", name)  
  
main()
```

What's your name? [user input]

NameError: name 'name' is not defined

"name" variable only exists in the defined function "main", not "hello".
This issue is called SCOPE. Scope is a variable that only exists in the context that has been defined. The variable "name" can only be used in the function "main".

"hello" only has a side-effect, and only prints out something to the screen via `print("hello", to)`. If we want "hello" to hand back a value, we use the "return" function to return a value explicitly back to me.

ex. assume user inputs [x]: 4

```
def main():  
    x = int(input("What's x? "))  
    print("x squared is", square(x))  
  
def square(n):  
    return n * n  
  
main()
```

What's x? [x]

x squared is 16

This allows me to pass the return value of the function "square" back to the function "main" rather than using a bunch of variables or calling functions within a function.